

# 数值算法 Homework 11

姓名: 雍崔扬

学号: 21307140051

Due: 08:00 May 12, 2025

## Problem 1

Show that the DFT matrix  $V_n$  diagonalizes the permutation matrix

$$J_n = \begin{bmatrix} 0 & 1 & & & \\ & 0 & 1 & & \\ & & \ddots & \ddots & \\ & & & 0 & 1 \\ 1 & & & & 0 \end{bmatrix}$$

i.e.  $V_n^{-1} J_n V_n$  is diagonal.

**Solution:**

考虑一维离散 Fourier 变换:

$$\tilde{f}_{\text{DFT}}[j] = \sum_{k=0}^{n-1} f[k] \exp\left(-\frac{\mathrm{i}2\pi jk}{n}\right) \quad (j = 0, 1, \dots, n-1)$$

$\Updownarrow$

$$f[k] = \frac{1}{n} \sum_{j=0}^{n-1} \tilde{f}_{\text{DFT}}[j] \cdot \exp\left(\frac{\mathrm{i}2\pi jk}{n}\right) \quad (k = 0, 1, \dots, n-1)$$

一维离散 Fourier 变换可表示为以下矩阵运算:

$$\vec{f}_{\text{DFT}} = \begin{bmatrix} \tilde{f}_{\text{DFT}}[0] \\ \tilde{f}_{\text{DFT}}[1] \\ \vdots \\ \tilde{f}_{\text{DFT}}[n-1] \end{bmatrix} = \begin{bmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \dots & \omega^{0 \cdot (n-1)} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \dots & \omega^{1 \cdot (n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{(n-1) \cdot 0} & \omega^{(n-1) \cdot 1} & \dots & \omega^{(n-1) \cdot (n-1)} \end{bmatrix} \begin{bmatrix} f[0] \\ f[1] \\ \vdots \\ f[n-1] \end{bmatrix} = V_n \cdot \vec{f}$$

$$\text{where } \omega := \exp\left(-\frac{\mathrm{i}2\pi}{n}\right)$$

$$\text{and } (V_n)_{j,k} = \exp\left(-\frac{\mathrm{i}2\pi jk}{n}\right) = \omega^{jk}$$

由于  $\omega^0, \omega^1, \dots, \omega^{n-1}$  互不相同, 故 Vandermonde 矩阵  $V_n \in \mathbb{C}^{n \times n}$  非奇异.

可以证明  $V_n^H V_n = \bar{V}_n V_n = n \cdot I_n$ :

$$\begin{aligned} (\bar{V}_n V_n)_{[j,k]} &= \sum_{l=0}^{n-1} \bar{V}_{j,l} \cdot V_{l,k} \\ &= \sum_{l=0}^{n-1} \omega^{-jl} \cdot \omega^{lk} \\ &= \sum_{l=0}^{n-1} \left[ \omega^{(k-l)} \right]^l \\ &= \begin{cases} n, & \text{if } k = l \\ \frac{1 - \omega^{(k-l)n}}{1 - \omega^{(k-l)}} = \frac{0}{1 - \omega^{(k-l)}} = 0, & \text{if } k \neq l \end{cases} \\ &= n \cdot \delta_{j,k} \end{aligned}$$

因此  $V_n^{-1} = \frac{1}{n} \overline{V}_n$

于是我们有:

$$\begin{aligned}
V_n^{-1} J_n V_n &= \frac{1}{n} \overline{V}_n J_n V_n \\
&= \frac{1}{n} \begin{bmatrix} \omega^{-0 \cdot 0} & \omega^{-0 \cdot 1} & \dots & \omega^{-0 \cdot (n-1)} \\ \omega^{-1 \cdot 0} & \omega^{-1 \cdot 1} & \dots & \omega^{-1 \cdot (n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{-(n-2) \cdot 0} & \omega^{-(n-2) \cdot 1} & \dots & \omega^{-(n-2) \cdot (n-1)} \\ \omega^{-(n-1) \cdot 0} & \omega^{-(n-1) \cdot 1} & \dots & \omega^{-(n-1) \cdot (n-1)} \end{bmatrix} \begin{bmatrix} 0 & 1 & & \\ & 0 & 1 & \\ & & \ddots & \ddots \\ & & & 0 & 1 \end{bmatrix} \begin{bmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \dots & \omega^{0 \cdot (n-1)} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \dots & \omega^{1 \cdot (n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{(n-2) \cdot 0} & \omega^{(n-2) \cdot 1} & \dots & \omega^{(n-2) \cdot (n-1)} \\ \omega^{(n-1) \cdot 0} & \omega^{(n-1) \cdot 1} & \dots & \omega^{(n-1) \cdot (n-1)} \end{bmatrix} \\
&= \frac{1}{n} \begin{bmatrix} \omega^{-0 \cdot 0} & \omega^{-0 \cdot 1} & \dots & \omega^{-0 \cdot (n-1)} \\ \omega^{-1 \cdot 0} & \omega^{-1 \cdot 1} & \dots & \omega^{-1 \cdot (n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{-(n-2) \cdot 0} & \omega^{-(n-2) \cdot 1} & \dots & \omega^{-(n-2) \cdot (n-1)} \\ \omega^{-(n-1) \cdot 0} & \omega^{-(n-1) \cdot 1} & \dots & \omega^{-(n-1) \cdot (n-1)} \end{bmatrix} \begin{bmatrix} \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \dots & \omega^{1 \cdot (n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{(n-2) \cdot 0} & \omega^{(n-2) \cdot 1} & \dots & \omega^{(n-2) \cdot (n-1)} \\ \omega^{(n-1) \cdot 0} & \omega^{(n-1) \cdot 1} & \dots & \omega^{(n-1) \cdot (n-1)} \end{bmatrix} \\
&= \frac{1}{n} \begin{bmatrix} \omega^{-0 \cdot 0} & \omega^{-0 \cdot 1} & \dots & \omega^{-0 \cdot (n-1)} \\ \omega^{-1 \cdot 0} & \omega^{-1 \cdot 1} & \dots & \omega^{-1 \cdot (n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{-(n-2) \cdot 0} & \omega^{-(n-2) \cdot 1} & \dots & \omega^{-(n-2) \cdot (n-1)} \\ \omega^{-(n-1) \cdot 0} & \omega^{-(n-1) \cdot 1} & \dots & \omega^{-(n-1) \cdot (n-1)} \end{bmatrix} \begin{bmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \dots & \omega^{0 \cdot (n-1)} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \dots & \omega^{1 \cdot (n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{(n-2) \cdot 0} & \omega^{(n-2) \cdot 1} & \dots & \omega^{(n-2) \cdot (n-1)} \\ \omega^{(n-1) \cdot 0} & \omega^{(n-1) \cdot 1} & \dots & \omega^{(n-1) \cdot (n-1)} \end{bmatrix} \\
&= \frac{1}{n} [\omega^{-jk}]_{j,k \in \{0, \dots, n-1\}} \cdot [\omega^{(j+1)k}]_{j,k \in \{0, \dots, n-1\}} \\
&= \frac{1}{n} \left[ \sum_{l=0}^{n-1} \omega^{-jl} \cdot \omega^{(l+1)k} \right]_{j,k \in \{0, \dots, n-1\}}
\end{aligned}$$

考虑  $V_n^{-1} J_n V_n$  的  $(j, k)$  元素:

(其中  $j, k \in \{0, 1, \dots, n-1\}$ )

$$\begin{aligned}
[V_n^{-1} J_n V_n]_{(j,k)} &= \frac{1}{n} \sum_{l=0}^{n-1} \omega^{-jl} \cdot \omega^{(l+1)k} \\
&= \frac{1}{n} \omega^k \sum_{l=0}^{n-1} \omega^{(k-j)l} \\
&= \frac{1}{n} \omega^k \sum_{l=0}^{n-1} [\omega^{(k-j)}]^l \\
&= \begin{cases} \frac{1}{n} \omega^k \cdot n = \omega^k, & \text{if } j = k \\ \frac{1}{n} \omega^k \frac{1 - \omega^{(k-j)n}}{1 - \omega^{(k-j)}} = \frac{1}{n} \omega^k \frac{0}{1 - \omega^{(k-j)}} = 0, & \text{if } j \neq k \end{cases}
\end{aligned}$$

因此  $V_n^{-1} J_n V_n$  为对角阵:

$$\begin{aligned}
V_n^{-1} J_n V_n &= \frac{1}{n} \left[ \sum_{l=0}^{n-1} \omega^{-jl} \cdot \omega^{(l+1)k} \right]_{j,k \in \{0, \dots, n-1\}} \\
&= \begin{bmatrix} \omega^0 & & & \\ & \omega^1 & & \\ & & \ddots & \\ & & & \omega^{n-2} & \\ & & & & \omega^{n-1} \end{bmatrix}
\end{aligned}$$

## Problem 2

For  $\vec{f} \in \mathbb{C}^n$ , its discrete Fourier transform  $\vec{f}_{\text{DFT}} = V_n \vec{f}$  is in general a complex vector.

When is  $\vec{f}_{\text{DFT}}$  real? Can you find a sufficient and necessary condition?

What about 2-D DFT?

- (Background)

考虑一维离散 Fourier 变换:

$$\begin{aligned}\tilde{f}_{\text{DFT}}[j] &= \sum_{k=0}^{n-1} f[k] \exp\left(-\frac{i2\pi jk}{n}\right) \quad (j = 0, 1, \dots, n-1) \\ &\Downarrow \\ f[k] &= \frac{1}{n} \sum_{j=0}^{n-1} \tilde{f}_{\text{DFT}}[j] \cdot \exp\left(\frac{i2\pi jk}{n}\right) \quad (k = 0, 1, \dots, n-1)\end{aligned}$$

一维离散 Fourier 变换可表示为以下矩阵运算:

$$\begin{aligned}\vec{f}_{\text{DFT}} = \begin{bmatrix} \tilde{f}_{\text{DFT}}[0] \\ \tilde{f}_{\text{DFT}}[1] \\ \vdots \\ \tilde{f}_{\text{DFT}}[n-1] \end{bmatrix} &= \begin{bmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \dots & \omega^{0 \cdot (n-1)} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \dots & \omega^{1 \cdot (n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{(n-1) \cdot 0} & \omega^{(n-1) \cdot 1} & \dots & \omega^{(n-1) \cdot (n-1)} \end{bmatrix} \begin{bmatrix} f[0] \\ f[1] \\ \vdots \\ f[n-1] \end{bmatrix} = V_n \cdot \vec{f} \\ &\text{where } \omega := \exp\left(-\frac{i2\pi}{n}\right) \\ &\text{and } (V_n)_{j,k} = \exp\left(-\frac{i2\pi jk}{n}\right) = \omega^{jk}\end{aligned}$$

二维离散 Fourier 变换如下:

$$\begin{aligned}\tilde{f}_{\text{DFT}}[n, m] &= \sum_{j=0}^{N-1} \sum_{k=0}^{M-1} f[j, k] \exp\left(-i2\pi \left(\frac{jn}{N} + \frac{km}{M}\right)\right) \\ &\text{where } n = 0, 1, \dots, N-1 \text{ and } m = 0, 1, \dots, M-1 \\ &\Downarrow \\ f[j, k] &= \frac{1}{NM} \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} \tilde{f}_{\text{DFT}}[n, m] \exp\left(i2\pi \left(\frac{jn}{N} + \frac{km}{M}\right)\right) \\ &\text{where } j = 0, 1, \dots, N-1 \text{ and } k = 0, 1, \dots, M-1\end{aligned}$$

和连续二维 Fourier 变换一样,

二维离散 Fourier 变换可以通过逐次一维离散 Fourier 变换得到, 即具有可分离性.

**Solution:**

首先考虑一维离散 Fourier 变换: (简单起见, 省略  $\vec{f}$  上的向量标记, 记为  $f$ )

$$\begin{aligned}\overline{V_n} \cdot \overline{f} &= \overline{f_{\text{DFT}}} = f_{\text{DFT}} = V_n \cdot f \\ &\Downarrow \\ f &= V_n^{-1} \cdot \overline{V_n} \cdot \overline{f} = \frac{1}{n} (\overline{V_n})^2 \overline{f}\end{aligned}$$

注意到:

$$\begin{aligned}
\left[(\overline{V}_n)^2\right]_{(j,k)} &= \sum_{l=0}^{n-1} \omega^{-jl} \cdot \omega^{-lk} \\
&= \sum_{l=0}^{n-1} \omega^{-l(j+k)} \\
&= \sum_{l=0}^{n-1} \left[\omega^{-(j+k)}\right]^l \\
&= \begin{cases} n, & \text{if } j+k = n \\ \frac{1-\omega^{-(j+k)n}}{1-\omega^{-(j+k)}}, & \text{if } j+k \neq n \end{cases}
\end{aligned}$$

因此  $\frac{1}{n}(\overline{V}_n)^2$  为反序矩阵:

$$\frac{1}{n}(\overline{V}_n)^2 = \begin{bmatrix} & & & & 1 \\ & & & 1 & \\ & & \cdots & & \\ & 1 & & & \\ 1 & & & & \end{bmatrix}$$

这说明 "离散 Fourier 变换  $f_{\text{DFT}}$  为实序列" 的充要条件是:

$$\begin{aligned}
f &= \frac{1}{n}(\overline{V}_n)^2 \bar{f} \\
&\Updownarrow \\
\begin{bmatrix} f[0] \\ f[1] \\ \vdots \\ f[n-2] \\ f[n-1] \end{bmatrix} &= \begin{bmatrix} & & & & 1 \\ & & & 1 & \\ & & \cdots & & \\ & 1 & & & \\ 1 & & & & \end{bmatrix} \begin{bmatrix} \overline{f[0]} \\ \overline{f[1]} \\ \vdots \\ \overline{f[n-2]} \\ \overline{f[n-1]} \end{bmatrix} = \begin{bmatrix} \overline{f[n-1]} \\ \overline{f[n-2]} \\ \vdots \\ \overline{f[1]} \\ \overline{f[0]} \end{bmatrix} \\
&\Updownarrow \\
f[k] &= \overline{f[n-k-1]} \quad (k = 0, 1, \dots, n-1)
\end{aligned}$$

这称为 **Hermite 条件**.

二维离散 Fourier 变换可以通过逐次一维离散 Fourier 变换得到, 即具有可分离性.

因此  $\{\tilde{f}_{\text{DFT}}[n, m]\}_{n=0, \dots, N-1}^{m=0, \dots, M-1}$  为实序列的充要条件是:

$$\begin{aligned}
f[n, m] &= \overline{f[N-n-1, M-m-1]} \\
&\text{where } n = 0, 1, \dots, N-1 \text{ and } m = 0, 1, \dots, M-1
\end{aligned}$$

## Problem 3

Implement the radix-2 FFT.

A non-recursive implementation is preferred.

Make sure that the complexity of your implementation is  $\Theta(N \log(N))$ .

**Solution:**

设  $N = 2^r$ , 考虑一维离散 Fourier 变换:

$$\begin{aligned}\tilde{f}_{\text{DFT}}[j] &= \sum_{k=0}^{N-1} f[k] \exp\left(-\frac{\mathrm{i}2\pi jk}{N}\right) \quad (j = 0, 1, \dots, N-1) \\ &\quad \Updownarrow \\ f[k] &= \frac{1}{N} \sum_{j=0}^{N-1} \tilde{f}_{\text{DFT}}[j] \cdot \exp\left(\frac{\mathrm{i}2\pi jk}{N}\right) \quad (k = 0, 1, \dots, N-1)\end{aligned}$$

我们将  $j, k$  记为二进制:

$$\begin{aligned}j &= [j_{r-1}, j_{r-2}, \dots, j_0] = 2^{r-1} \cdot j_{r-1} + \dots + 2 \cdot j_1 + j_0 \\ k &= [k_{r-1}, k_{r-2}, \dots, k_0] = 2^{r-1} \cdot k_{r-1} + \dots + 2 \cdot k_1 + k_0\end{aligned}$$

于是我们有:

$$\begin{aligned}\tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_0]) &= \tilde{f}_{\text{DFT}}[j] \\ &= \sum_{k=0}^{N-1} f[k] \exp\left(-\frac{\mathrm{i}2\pi jk}{N}\right) \\ &= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, k_{r-2}, \dots, k_0]) \omega^{jk} \\ &= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, k_{r-2}, \dots, k_0]) \omega^{2^{r-1}jk_{r-1}} \cdot \omega^{2^{r-2}jk_{r-2}} \cdots \omega^{2^0jk_0}\end{aligned}$$

其中  $\omega = \exp(-\mathrm{i}2\pi/N)$

注意到:

$$\begin{aligned}\omega^{2^{r-1}jk_{r-1}} &= \omega^{2^{r-1}(j_0+2j_1+\dots+2^{r-1}j_{r-1}) \cdot k_{r-1}} \\ &= \omega^{2^{r-1}j_0k_{r-1}} \cdot \omega^{2^rj_1k_{r-1}} \cdots \omega^{2^{2r-2}j_{r-1}k_{r-1}} \\ &\quad (\text{note that } \omega^{2^r \cdot l} = \omega^{N \cdot l} = 1^l = 1 \text{ for all integer } l) \\ &= \omega^{2^{r-1}j_0k_{r-1}} \cdot 1 \cdots 1 \\ &= \omega^{2^{r-1}j_0k_{r-1}} \\ \hline \omega^{2^{r-2}jk_{r-2}} &= \omega^{2^{r-2}(j_0+2j_1+\dots+2^{r-1}j_{r-1}) \cdot k_{r-2}} \\ &= \omega^{2^{r-2}j_0k_{r-2}} \cdot \omega^{2^{r-1}j_1k_{r-2}} \cdots \omega^{2^{2r-3}j_{r-1}k_{r-2}} \\ &\quad (\text{note that } \omega^{2^r \cdot l} = \omega^{N \cdot l} = 1^l = 1 \text{ for all integer } l) \\ &= \omega^{2^{r-2}j_0k_{r-2}} \cdot \omega^{2^{r-1}j_1k_{r-2}} \cdot 1 \cdots 1 \\ &= \omega^{2^{r-2}(j_0+2j_1)k_{r-2}} \\ \hline \omega^{2^{r-p}jk_{r-p}} &= \omega^{2^{r-p}(j_0+2j_1+\dots+2^{p-1}j_{p-1})k_{r-p}} \text{ for } p = 1, 2, \dots, r\end{aligned}$$

因此我们有:

$$\begin{aligned}\tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_0]) &= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, k_{r-2}, \dots, k_0]) \omega^{2^{r-1}jk_{r-1}} \cdot \omega^{2^{r-2}jk_{r-2}} \cdots \omega^{2^0jk_0} \\ &= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, k_{r-2}, \dots, k_0]) \omega^{2^{r-1}j_0k_{r-1}} \cdot \omega^{2^{r-2}(j_0+2j_1)k_{r-2}} \cdots \omega^{2^0(j_0+2j_1+\dots+2^{r-1}j_{r-1})k_0}\end{aligned}$$

利用分次求和, 并对中间结果标号, 即有:

$$\begin{aligned}
f_1([j_0, k_{r-2}, \dots, k_1, k_0]) &= \sum_{k_{r-1}=0}^1 f([k_{r-1}, k_{r-2}, \dots, k_1, k_0]) \omega^{2^{r-1} j_0 k_{r-1}} \\
&= f([0, k_{r-2}, \dots, k_1, k_0]) \cdot \omega^{2^{r-1} j_0 \cdot 0} + f([1, k_{r-2}, \dots, k_1, k_0]) \cdot \omega^{2^{r-1} j_0 \cdot 1} \\
&= f([0, k_{r-2}, \dots, k_1, k_0]) + f([1, k_{r-2}, \dots, k_1, k_0]) \cdot \omega^{2^{r-1} j_0}
\end{aligned}$$

$$\begin{aligned}
f_2([j_0, j_1, k_{r-3}, \dots, k_1, k_0]) &= \sum_{k_{r-2}=0}^1 f_1([j_0, k_{r-2}, k_{r-3}, \dots, k_1, k_0]) \omega^{2^{r-2} (j_0 + 2j_1) k_{r-2}} \\
&= f_1([j_0, 0, k_{r-3}, \dots, k_1, k_0]) \cdot \omega^{2^{r-2} (j_0 + 2j_1) \cdot 0} + f_1([j_0, 1, k_{r-3}, \dots, k_1, k_0]) \cdot \omega^{2^{r-2} (j_0 + 2j_1) \cdot 1} \\
&= f_1([j_0, 0, k_{r-3}, \dots, k_1, k_0]) + f_1([j_0, 1, k_{r-3}, \dots, k_1, k_0]) \cdot \omega^{2^{r-2} (j_0 + 2j_1)}
\end{aligned}$$

⋮

$$\begin{aligned}
f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}]) &= \sum_{k_0=0}^1 f_{r-1}([j_0, j_1, \dots, j_{r-2}, k_0]) \omega^{(j_0 + 2j_1 + \dots + 2^{r-1} j_{r-1}) k_0} \\
&= f_{r-1}([j_0, j_1, \dots, j_{r-2}, 0]) \omega^{(j_0 + 2j_1 + \dots + 2^{r-1} j_{r-1}) \cdot 0} + f_{r-1}([j_0, j_1, \dots, j_{r-2}, 1]) \omega^{(j_0 + 2j_1 + \dots + 2^{r-1} j_{r-1}) \cdot 1} \\
&= f_{r-1}([j_0, j_1, \dots, j_{r-2}, 0]) + f_{r-1}([j_0, j_1, \dots, j_{r-2}, 1]) \cdot \omega^{(j_0 + 2j_1 + \dots + 2^{r-1} j_{r-1})} \\
\text{finally } \tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_1, j_0]) &= f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}])
\end{aligned}$$

上述递推公式正是 Cooley 和 Tukey 提出的 FFT 算法的原始形式。

它共有  $r$  重和式，每重和式需要  $N$  次复数乘法法和  $N$  次复数加法，

因此总运算量为  $Nr$  次复数乘法法和  $Nr$  次复数加法。

如果考虑到对称性  $\omega^{l+N/2} = -\omega^l$ ，则复数乘法的运算次数还可减少一半。

**Cooley-Tukey 算法** (Decimation in Time, DIT) 采用了解析指标  $k$  的办法。

若采用解析指标  $j$  的办法，则导出的 FFT 算法称为 **Sande-Tukey 算法** (Decimation in Frequency, DIF)

$$\begin{aligned}
\tilde{f}_{\text{DFT}}([j_{r-1}, \dots, j_1, j_0]) &= \tilde{f}_{\text{DFT}}[j] \\
&= \sum_{k=0}^{N-1} f[k] \exp\left(-\frac{i2\pi jk}{N}\right) \\
&= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, \dots, k_1, k_0]) \omega^{jk} \\
&= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, \dots, k_1, k_0]) \omega^{2^{r-1} j_{r-1} k} \cdot \omega^{2^{r-2} j_{r-2} k} \cdots \omega^{2^0 j_0 k} \\
&= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, \dots, k_1, k_0]) \omega^{2^{r-1} j_{r-1} k_0} \cdot \omega^{2^{r-2} j_{r-2} (k_0 + 2k_1)} \cdots \omega^{2^0 j_0 (k_0 + 2k_1 + \cdots + 2^{r-1} k_{r-1})}
\end{aligned}$$

Sande-Tukey 算法的迭代公式如下：

$$\begin{aligned}
f_1([j_0, k_{r-2}, \dots, k_1, k_0]) &= \sum_{k_{r-1}=0}^1 f([k_{r-1}, k_{r-2}, \dots, k_1, k_0]) \omega^{2^0 j_0 (k_0 + 2k_1 + \dots + 2^{r-2} k_{r-2} + 2^{r-1} k_{r-1})} \\
&= f([0, k_{r-2}, \dots, k_1, k_0]) \cdot \omega^{2^0 j_0 (k_0 + 2k_1 + \dots + 2^{r-2} k_{r-2} + 2^{r-1} \cdot 0)} \\
&\quad + f([1, k_{r-2}, \dots, k_1, k_0]) \cdot \omega^{2^0 j_0 (k_0 + 2k_1 + \dots + 2^{r-2} k_{r-2} + 2^{r-1} \cdot 1)} \\
\hline
f_2([j_0, j_1, k_{r-3}, \dots, k_1, k_0]) &= \sum_{k_{r-2}=0}^1 f_1([j_0, k_{r-2}, k_{r-3}, \dots, k_1, k_0]) \omega^{2^1 j_1 (k_0 + 2k_1 + \dots + 2^{r-3} k_{r-3} + 2^{r-2} k_{r-2})} \\
&= f_1([j_0, 0, k_{r-3}, \dots, k_1, k_0]) \cdot \omega^{2^1 j_1 (k_0 + 2k_1 + \dots + 2^{r-3} k_{r-3} + 2^{r-2} \cdot 0)} \\
&\quad + f_1([j_0, 1, k_{r-3}, \dots, k_1, k_0]) \cdot \omega^{2^1 j_1 (k_0 + 2k_1 + \dots + 2^{r-3} k_{r-3} + 2^{r-2} \cdot 1)} \\
\hline
&\vdots \\
\hline
f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}]) &= \sum_{k_0=0}^1 f_{r-1}([j_0, j_1, \dots, j_{r-2}, k_0]) \omega^{2^{r-1} j_{r-1} k_0} \\
&= f_{r-1}([j_0, j_1, \dots, j_{r-2}, 0]) \omega^{2^{r-1} j_{r-1} \cdot 0} + f_{r-1}([j_0, j_1, \dots, j_{r-2}, 1]) \omega^{2^{r-1} j_{r-1} \cdot 1} \\
&\text{finally } \tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_1, j_0]) = f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}])
\end{aligned}$$

从程序设计的角度来讲，Sande-Tukey 算法具有更好的效率。

因为计算过程中  $\omega$  的幂次是降序的，而 Cooley-Tukey 算法则是跳跃的。

不过两者的运算量是一样的。

Matlab 代码:

```
function y = bitrevorder(x)
    % In-place bit-reversal permutation of vector x
    n = numel(x);
    r = log2(n);
    if floor(r) ~= r
        error('bitrevorder: length must be power of 2');
    end
    y = zeros(size(x));
    for k = 0:n-1
        % reverse bits of k
        b = dec2bin(k, r);
        b_rev = b(end:-1:1);
        j = bin2dec(b_rev);
        y(j+1) = x(k+1);
    end
end

function x = fft_radix2_Coooley_Tukey(x)
    % Cooley-Tukey, decimation-in-time, non-recursive
    N = length(x);
    if mod(N,2) ~= 0 || log2(N) ~= floor(log2(N))
        error('fft_radix2_dit: N must be 2^r');
    end
    x = bitrevorder(x);
    stages = log2(N);
    for s = 1:stages
        m = 2^s;
        half = m/2;
        w = exp(-2i*pi*(0:half-1)/m);
        for k = 1:m:N
            for j = 0:half-1
                t = w(j+1)*x(k+j+half);
```

```

        u = x(k+j);
        x(k+j)      = u + t;
        x(k+j+half) = u - t;
    end
end
end
X = x;
end

function X = fft_radix2_Sande_Tukey(x)
% Sande-Tukey, decimation-in-frequency, non-recursive
N = length(x);
if mod(N,2)~=0 || log2(N)~=floor(log2(N))
    error('fft_radix2_dif: N must be 2^r');
end
stages = log2(N);
for s = stages:-1:1
    m = 2^s;
    half = m/2;
    w = exp(-2i*pi*(0:half-1)/m);
    for k = 1:m:N
        for j = 0:half-1
            u = x(k+j);
            t = x(k+j+half);
            x(k+j)      = u + t;
            x(k+j+half) = (u - t) * w(j+1);
        end
    end
end
X = bitrevorder(x);
end

```

函数调用:

```

% Compare radix-2 Cooley-Tukey (DIT) vs. Sande-Tukey (DIF) vs. built-in fft
clearvars; close all; rng(42);

% list of test lengths (some powers of two, some not)
N_list = [1024, 2048, 4096, 8192, 16384, 32768, 65536, 131072];
num_tests = numel(N_list);

t_dit = zeros(1,num_tests);
t_dif = zeros(1,num_tests);
t_mat = zeros(1,num_tests);
res_dit = zeros(1,num_tests);
res_dif = zeros(1,num_tests);

for idx = 1:num_tests
    N0 = N_list(idx);
    % next power of two
    r = ceil(log2(N0));
    N = 2^r;

    % random complex signal of original length, pad with zeros
    x0 = randn(N0,1) + 1i*randn(N0,1);
    x = [x0; zeros(N-N0,1)];

```



```

% MATLAB fft
tic; X_mat = fft(x); t_mat(idx) = toc;

% DIT implementation
tic; X_dit = fft_radix2_Cooley_Tukey(x); t_dit(idx) = toc;

% DIF implementation
tic; X_dif = fft_radix2_Sande_Tukey(x); t_dif(idx) = toc;

% residuals (Frobenius norm on length-N vector = L2 norm)
res_dit(idx) = norm(X_dit - X_mat, 'fro');
res_dif(idx) = norm(X_dif - X_mat, 'fro');

fprintf('N0=%5d -> N=%5d | t_mat=%.4f t_dit=%.4f t_dif=%.4f | r_dit=%.1e r_dif=%.1e\n', ...
        N0, N, t_mat(idx), t_dit(idx), t_dif(idx), res_dit(idx), res_dif(idx));
end

% Plot timing on log-log
figure; hold on; grid on;
loglog(N_list, t_mat, 's-', 'DisplayName', 'MATLAB fft');
loglog(N_list, t_dit, 'o-', 'DisplayName', 'DIT (Cooley-Tukey)');
loglog(N_list, t_dif, 'x-', 'DisplayName', 'DIF (Sande-Tukey)');
xlabel('Signal length N'); ylabel('Time (seconds)');
title('Computation time vs. N (log-log scale)');
legend('Location', 'northwest');

% Plot residuals
figure; hold on; grid on;
semilogy(N_list, res_dit, 'o-', 'DisplayName', 'r_{DIT}');
semilogy(N_list, res_dif, 'x-', 'DisplayName', 'r_{DIF}');
xlabel('Signal length N'); ylabel('Frobenius residual');
title('Residual vs. N');
legend('Location', 'northeast');

% Empirical complexity fit: time  $\approx C \cdot (N \log_2 N)^p$  for DIT
Xv = N_list .* log2(N_list);
p = polyfit(log(Xv), log(t_dit), 1);
fprintf('\nEmpirical exponent for DIT: p  $\approx$  %.3f (ideal = 1)\n', p(1));

% Overlay fit line
figure; hold on; grid on;
scatter(log(Xv), log(t_dit), 'filled');
plot(log(Xv), polyval(p, log(Xv)), 'Linewidth', 1.5);
xlabel('log(N·log2 N)'); ylabel('log(time)');
title(sprintf('Empirical fit: log(time) = %.2f·log(N·log2 N) + %.2f', p(1), p(2)));
legend('data', 'fit', 'Location', 'best');

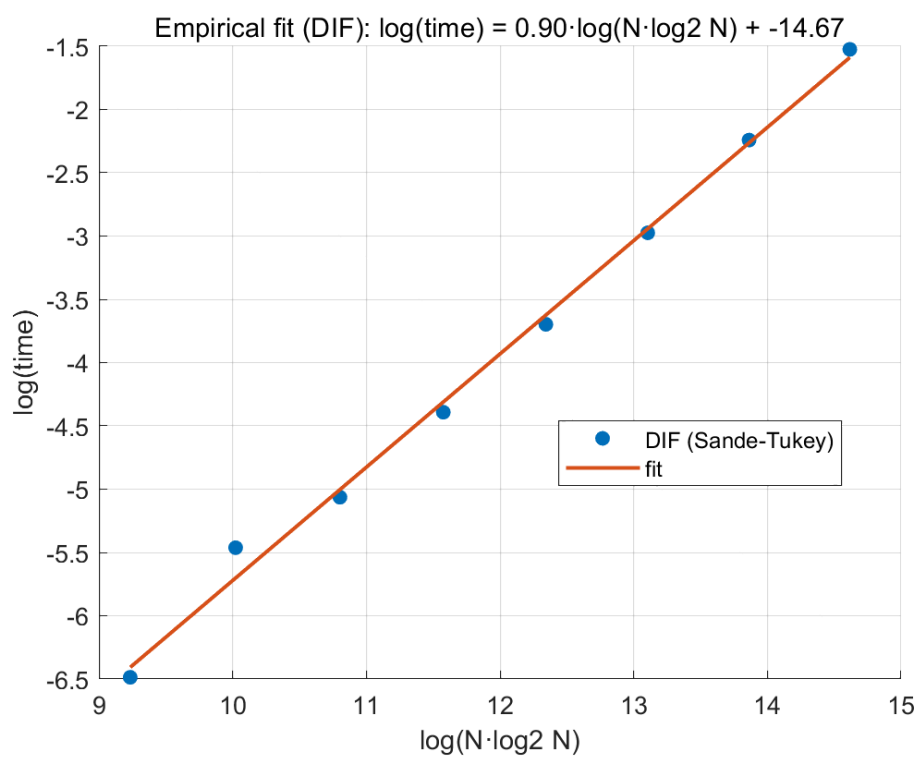
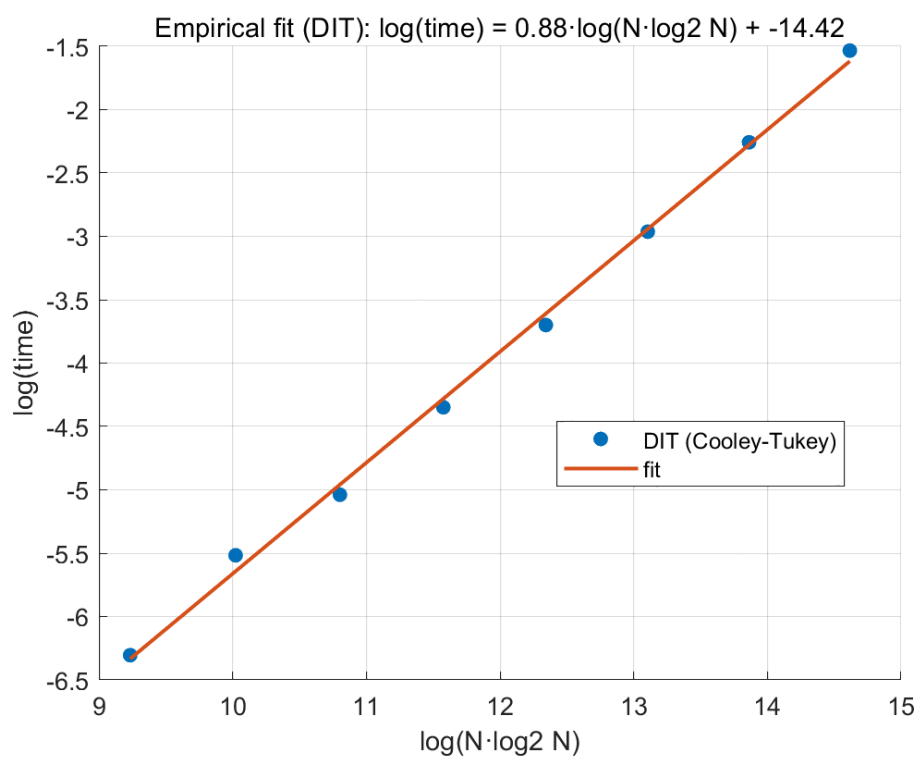
% Empirical complexity fit: time  $\approx C \cdot (N \log_2 N)^p$  for DIF
Xv_dif = N_list .* log2(N_list);
p_dif = polyfit(log(Xv_dif), log(t_dif), 1);
fprintf('Empirical exponent for DIF: p  $\approx$  %.3f (ideal = 1)\n', p_dif(1));

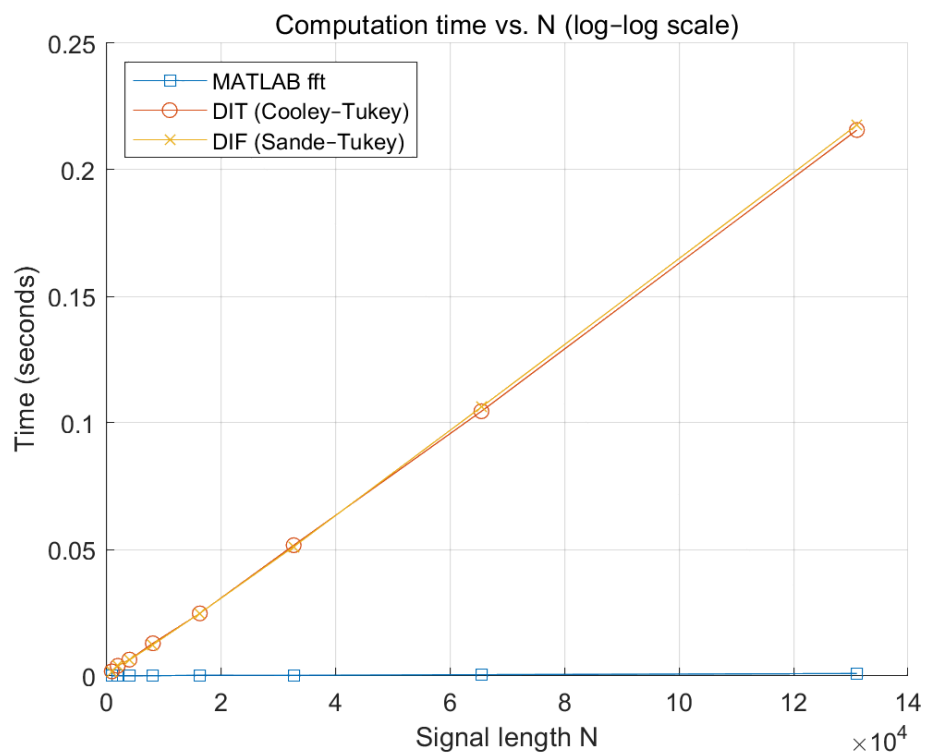
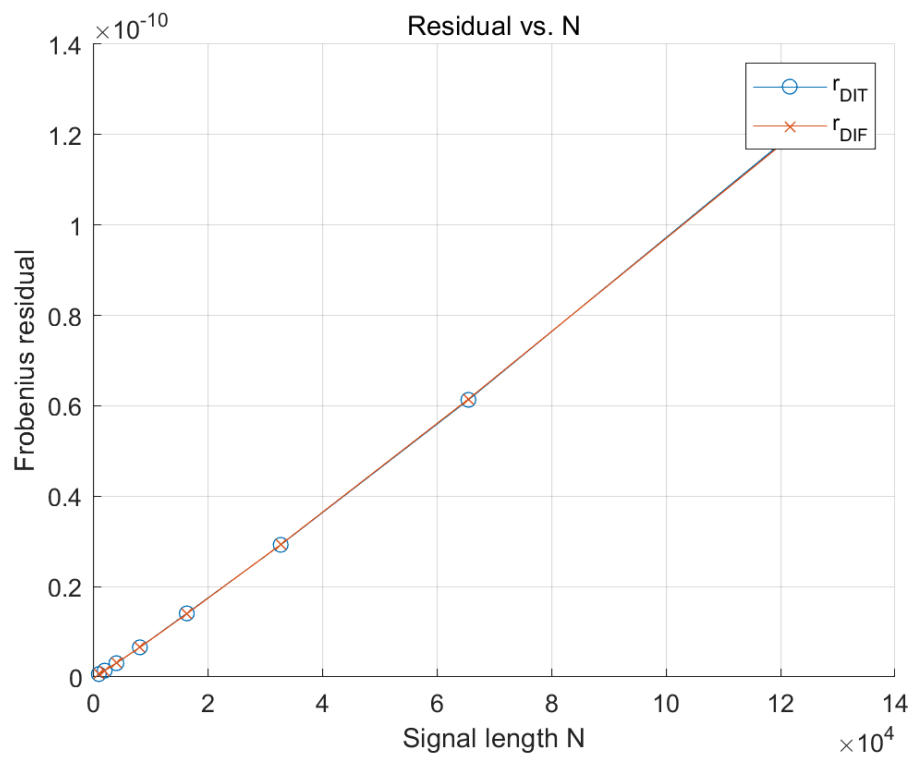
% Overlay fit line for DIF
figure; hold on; grid on;
scatter(log(Xv_dif), log(t_dif), 'filled');
plot(log(Xv_dif), polyval(p_dif, log(Xv_dif)), 'Linewidth', 1.5);
xlabel('log(N·log2 N)'); ylabel('log(time)');
title(sprintf('Empirical fit (DIF): log(time) = %.2f·log(N·log2 N) + %.2f', p_dif(1), p_dif(2)));

```

```
legend('data','fit','Location','best');
```

运行结果:





## Problem 4

Use trigonometric polynomials up to degree  $N$  to interpolate the periodic square wave

$$f(x) = \begin{cases} 1, & \text{if } x \in (2n, 2n+1) \\ 0, & \text{if } x \text{ is an integer} \\ -1, & \text{if } x \in (2n-1, 2n) \end{cases}$$

with equispaced interpolation nodes.

Make plots for a few different values of  $N$ .

What happens if  $N$  becomes large?

**Solution:**

设  $f(t)$  是定义在实数域上的周期为  $T$  的复值函数.

给定  $[0, T]$  上的  $2N + 1$  个等距节点:

$$t_j = jh \quad (j = 0, 1, \dots, 2N)$$

$$\text{where } h := \frac{T}{2N + 1}$$

假设三角插值问题的解具有如下形式:

$$\begin{aligned} T_N(t) &= \frac{\tilde{a}_0}{2} + \sum_{k=1}^N \left[ \tilde{a}_k \cos\left(\frac{2k\pi t}{T}\right) + \tilde{b}_k \sin\left(\frac{2k\pi t}{T}\right) \right] \\ &= \sum_{k=-N}^N \tilde{c}_k \exp\left(\frac{i2k\pi t}{T}\right) \end{aligned}$$

根据插值条件  $T_N(t_j) = f(t_j)$  ( $j = 0, 1, \dots, 2N$ ) 得到的 Vandermonde 系统如下:

$$\vec{f} = \begin{bmatrix} f(t_0) \\ f(t_1) \\ \vdots \\ f(t_{2N}) \end{bmatrix} = \begin{bmatrix} T_N(t_0) \\ T_N(t_1) \\ \vdots \\ T_N(t_{2N}) \end{bmatrix} = \begin{bmatrix} \omega^0 & \omega^0 & \cdots & \omega^0 \\ \omega^{-N} & \omega^{-N+1} & \cdots & \omega^N \\ \vdots & \vdots & \vdots & \vdots \\ \omega^{2N \cdot (-N)} & \omega^{2N \cdot (-N+1)} & \cdots & \omega^{2N \cdot N} \end{bmatrix} \begin{bmatrix} \tilde{c}_{-N} \\ \tilde{c}_{-N+1} \\ \vdots \\ \tilde{c}_N \end{bmatrix} = V \cdot \vec{c}$$

where  $\omega := \exp\left(\frac{i2\pi}{2N+1}\right)$

$$\text{and } V_{j,k} = \exp\left(\frac{i2k\pi \cdot t_j}{T}\right) = \exp\left(\frac{i2k\pi}{T} \cdot \frac{jT}{2N+1}\right) = \exp\left(jk \cdot \frac{i2\pi}{2N+1}\right) = \omega^{jk}$$

由于  $\omega^{-N}, \omega^{-N+1}, \dots, \omega^N$  互不相同, 故 Vandermonde 矩阵  $V \in \mathbb{C}^{(2N+1) \times (2N+1)}$  非奇异.

因此三角插值问题的解存在且唯一, 其系数为:

$$\begin{aligned} \tilde{a}_k &= \frac{2}{2N+1} \sum_{j=0}^{2N} f(t_j) \cdot \operatorname{Re}(\omega^{-jk}) \quad (\text{note that } \operatorname{Re}(\omega^{-jk}) = \operatorname{Re}(\omega^{jk})) \\ &= \frac{2}{2N+1} \sum_{j=0}^{2N} f(t_j) \cdot \operatorname{Re}(\omega^{jk}) \\ &= \frac{2}{2N+1} \sum_{j=0}^{2N} f\left(\frac{jT}{2N+1}\right) \cos\left(\frac{2jk\pi}{2N+1}\right) \quad (k = 0, 1, \dots, N) \\ \tilde{b}_k &= \frac{2}{2N+1} \sum_{j=0}^{2N} f(t_j) \cdot [-\operatorname{Im}(\omega^{-jk})] \quad (\text{note that } \operatorname{Im}(\omega^{-jk}) = -\operatorname{Im}(\omega^{jk})) \\ &= \frac{2}{2N+1} \sum_{j=0}^{2N} f(t_j) \cdot \operatorname{Im}(\omega^{jk}) \\ &= \frac{2}{2N+1} \sum_{j=0}^{2N} f\left(\frac{jT}{2N+1}\right) \sin\left(\frac{2jk\pi}{2N+1}\right) \quad (k = 1, 2, \dots, N) \\ \hline \tilde{c}_k &= \frac{2}{2N+1} \sum_{j=0}^{2N} f(t_j) \cdot \omega^{-jk} \\ &= \frac{1}{2N+1} \sum_{j=0}^{2N} f\left(\frac{jT}{2N+1}\right) \exp\left(-\frac{i2jk\pi}{2N+1}\right) \quad (k = 0, \pm 1, \dots, \pm N) \end{aligned}$$

有趣的是,  $\tilde{a}_k, \tilde{b}_k, \tilde{c}_k$  可通过对  $a_k, b_k, c_k$  运用复合矩形求积公式做数值积分得到, 节点为  $t_j = \frac{jT}{2N+1}$  ( $j = 0, 1, \dots, 2N+1$ ), 其中  $a_k, b_k, c_k$  为:

$$a_k = \frac{2}{T} \int_{-T/2}^{T/2} f(t) \cos\left(\frac{2k\pi t}{T}\right) dt$$

$$b_k = \frac{2}{T} \int_{-T/2}^{T/2} f(t) \sin\left(\frac{2k\pi t}{T}\right) dt$$

$$c_k = \frac{1}{T} \int_{-T/2}^{T/2} f(t) \exp\left(-\frac{i2k\pi t}{T}\right) dt$$

Matlab 代码:

```
% 用三角多项式插值周期方波, 展示不同次数 d 的逼近效果
clearvars; close all;

% 基本参数
T = 2*pi;           % 周期
M_plot = 1000;       % 绘图用的细分点数
t_plot = linspace(0, T, M_plot+1); t_plot(end)=[]; % [0,2π) 上的绘图点

% 定义方波 f(t)
f = @(t) 1 .* (mod(t,2*pi)<pi) + (-1) .* (mod(t,2*pi)>=pi);

% 一组选取的插值次数 d
N_list = [4, 16, 32, 128];

figure('Position',[100 100 800 600]);
for p = 1:length(N_list)
    N = N_list(p);
    J = 2*N+1;
    t_nodes = (0:J-1)' * (T/J); % equispaced nodes on [0,2π)
    y_nodes = f(t_nodes); % 方波取值

    % 计算复系数 c_k, k=-N..N
    k = (-N:N).'; % 列向量
    % 构造指数矩阵 exp(-i*k*t_j)
    E = exp(-1i * (k * t_nodes.') * (2*pi/T));
    c = (1/J) * (E * y_nodes); % (2N+1)x1

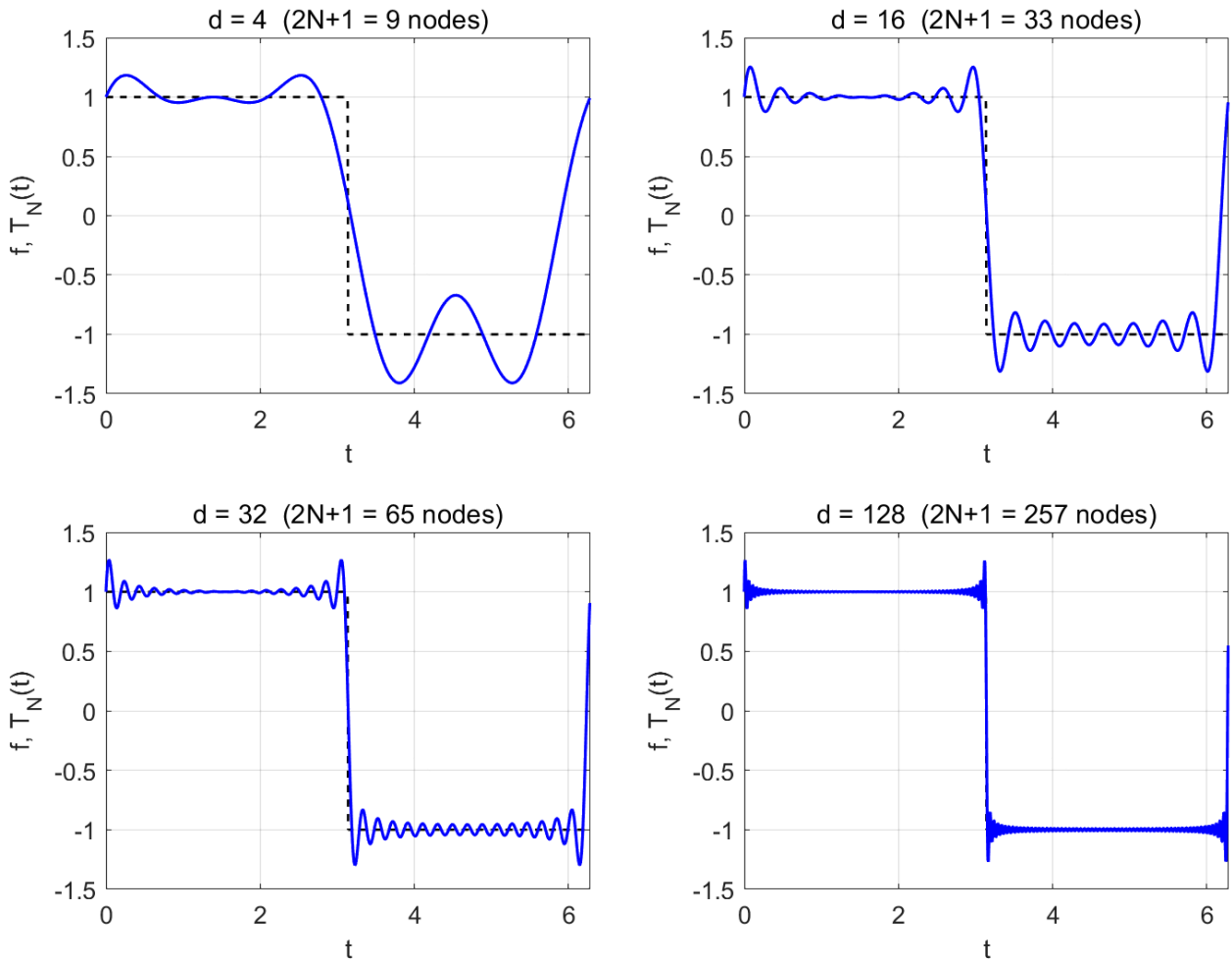
    % 在细分点 t_plot 上重建插值多项式
    % T_N(t) = sum_{k=-N..N} c_k exp(i*k*t)
    % 向量化计算
    E_plot = exp(1i * (k * t_plot) * (2*pi/T)); % (2N+1)xM_plot
    T_N_plot = real(c.' * E_plot); % 1 x M_plot

    % 绘图
    subplot(2,2,p);
    plot(t_plot, f(t_plot), 'k--', 'Linewidth',1); hold on;
    plot(t_plot, T_N_plot, 'b', 'Linewidth',1.2);
    xlabel('t'); ylabel('f, T_N(t)');
    title(sprintf('d = %d (2N+1 = %d nodes)', N, J));
    axis([0 T -1.5 1.5]);
    grid on;
end

sgtitle('三角插值: 周期方波的不同次数逼近', 'FontSize',14);
```

运行结果:

### 三角插值：周期方波的不同次数逼近



**Gibbs 现象**是指在使用有限项的三角插值多项式来逼近具有跳跃不连续点的周期函数时，逼近函数在不连续点附近出现的**过冲** (overshoot) 和**欠冲** (undershoot) 现象。

即使增加多项式的阶数，这种振荡行为仍然存在，且在不连续点附近不会完全消失。

对于  $f(\cdot)$  的不连续点  $t_*$ ，记  $c := f(t_*^+) - f(t_*^-)$ ，则我们有：

$$\lim_{N \rightarrow \infty} \max_{x \rightarrow x_0^\pm} |T_N(t) - f(t)| = \gamma \cdot c$$

其中 Gibbs 常数  $\gamma \approx 0.08949$ 。

- **振荡幅度**: 随着多项式阶数  $N$  的增加，过冲和欠冲的幅度趋于稳定，约为跳跃幅度的 9%。
- **振荡区域宽度**: 振荡主要集中在不连续点附近，且其宽度随着  $N$  的增加而变窄。
- **收敛性质**: 在不连续点处，逼近函数  $T_N(t)$  收敛到  $f(t)$  左右极限的平均值，而不是函数在该点的实际值。

## Problem 5

You are given an audio file DTMF [dialing.ogg](#), which contains 80 touch tones from a DTMF keyboard.

Try to determine the keys corresponding to the tones according to the following table.

|        | 1209 Hz | 1336 Hz | 1477 Hz | 1633 Hz |
|--------|---------|---------|---------|---------|
| 697 Hz | 1       | 2       | 3       | A       |
| 770 Hz | 4       | 5       | 6       | B       |
| 852 Hz | 7       | 8       | 9       | C       |
| 941 Hz | *       | 0       | #       | D       |

```
% ===== DTMF 频率参考表 =====
low_freqs = [697, 770, 852, 941];
high_freqs = [1209, 1336, 1477, 1633];
dtmf_keys = ['1', '2', '3', 'A';
             '4', '5', '6', 'B';
             '7', '8', '9', 'C';
             '*', '0', '#', 'D'];
```

## (1) Segmentation

首先将 DTMF 信号分为 8 段:

```
% 主程序: 读取信号并自动检测块, 然后使用自定义 FFT 解码每个 DTMF 信号
[signal, fs] = audioread('DTMF_dialing.ogg');
signal = signal(:, 1); % 如果是立体声, 取一个通道

% ===== 自动分段部分 =====
win_len = round(0.02 * fs); % 20ms window
energy = zeros(length(signal), 1);
for i = win_len:length(signal)
    window = signal(i - win_len + 1:i);
    energy(i) = sqrt(mean(window.^2));
end
energy_smooth = movmean(energy, round(0.01 * fs));
threshold = mean(energy_smooth) + 0.5 * std(energy_smooth);
isTone = energy_smooth > threshold;
d = diff([0; isTone; 0]);
start_idx = find(d == 1);
end_idx = find(d == -1) - 1;

% 每块包含10个DTMF信号, 共8块
numPerBlock = 10;
numBlocks = length(start_idx) / numPerBlock;
if mod(length(start_idx), 10) ~= 0
    error("检测到的 DTMF 音段数量不是 10 的整数倍!");
end

block = zeros(numBlocks, 2);
for b = 1:numBlocks
    i1 = (b - 1) * numPerBlock + 1;
    i2 = b * numPerBlock;
    block(b, 1) = start_idx(i1);
    block(b, 2) = end_idx(i2);
end

% 显示每个块的样本范围
```

```

disp("Detected block segmentation (sample indices):");
disp(block);

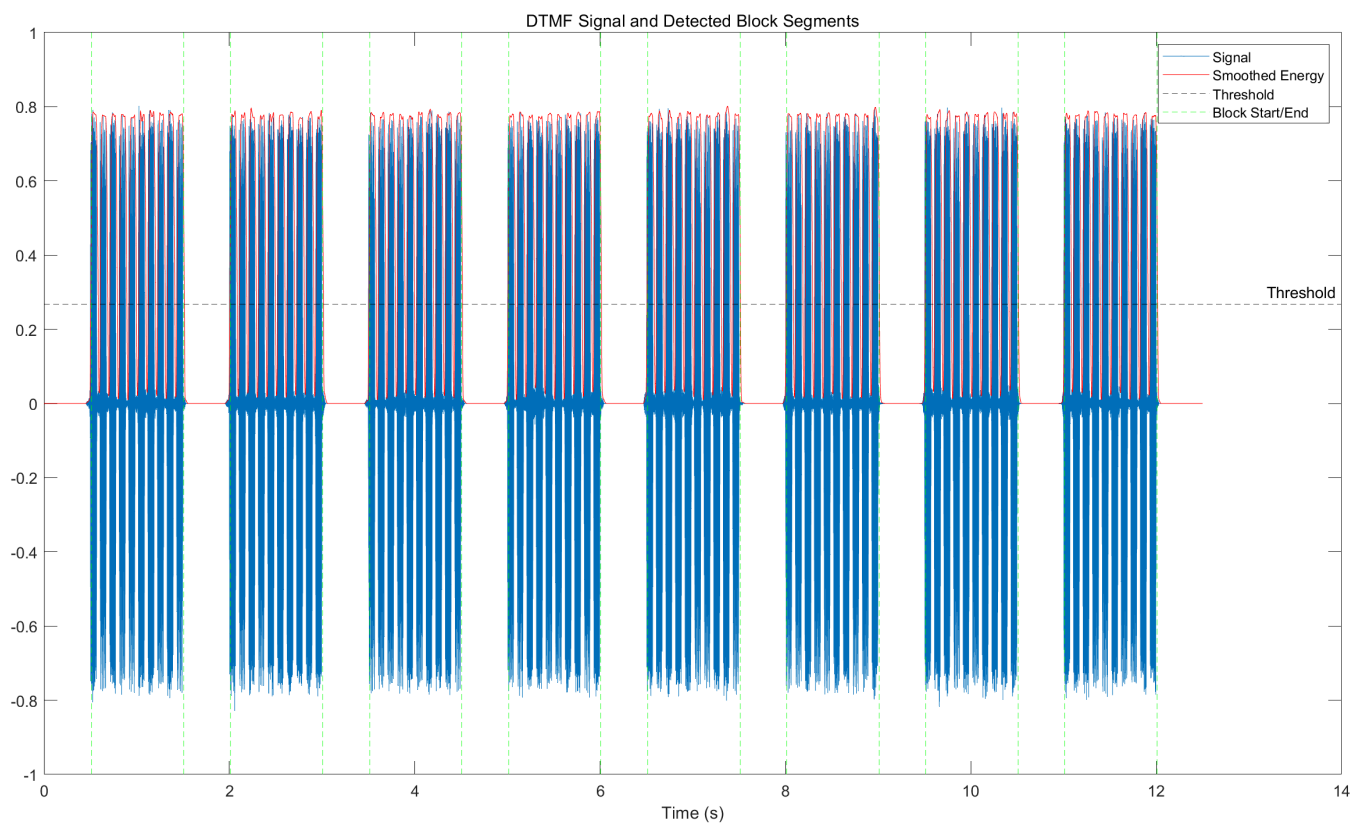
% ===== 可视化 =====
t = (1:length(signal)) / fs;
figure;
plot(t, signal);
hold on;
plot(t, energy_smooth / max(energy_smooth) * max(signal), 'r');
yline(threshold, '--k', 'Threshold');
for i = 1:numBlocks
    xline(block(i,1)/fs, 'g--');
    xline(block(i,2)/fs, 'g--');
end
title('DTMF Signal and Detected Block Segments');
xlabel('Time (s)');
legend('Signal', 'Smoothed Energy', 'Threshold', 'Block Start/End');

```

运行结果:

Detected block segmentation (sample indices):

|       |       |
|-------|-------|
| 4101  | 12065 |
| 16099 | 24065 |
| 28101 | 36065 |
| 40101 | 48064 |
| 52102 | 60065 |
| 64101 | 72062 |
| 76100 | 84064 |
| 88104 | 96067 |





## (2) FFT

使用本次作业 Problem 3 给出的快速 Fourier 变换算法 `fft_radix2_Coolley_Tukey`:

```
% ===== 解码每个信号块 =====
% 逐块解码并统一输出
fprintf("\n===== 解码结果 =====\n");
for b = 1:numBlocks
    segment = signal(block(b,1):block(b,2));
    seg_starts = linspace(1, length(segment), numPerBlock+1);

    keys_block = repmat(' ', 1, numPerBlock * 2); % 存储本块的10个按键信号
    for i = 1:numPerBlock
        idx1 = floor(seg_starts(i));
        idx2 = floor(seg_starts(i+1)) - 1;
        tone = segment(idx1:idx2);

        % 补零至 2 的幂长
        N = 2^nextpow2(length(tone));
        tone_pad = [tone; zeros(N - length(tone), 1)];

        % 自定义 FFT 计算
        x = fft_radix2_Coolley_Tukey(tone_pad);
        f_axis = (0:N-1) * fs / N;
        mag = abs(X);
        half_spectrum = mag(1:floor(N/2));
        f_axis_half = f_axis(1:floor(N/2));

        % 在低频和高频段内寻找最大幅值点
        [~, low_idx_found] = max(arrayfun(@(f) ...
            max(half_spectrum(abs(f_axis_half - f) < 20)), low_freqs));
        [~, high_idx_found] = max(arrayfun(@(f) ...
            max(half_spectrum(abs(f_axis_half - f) < 20)), high_freqs));

        % 找到按键字符
        key = dtmf_keys(low_idx_found, high_idx_found);
        keys_block(2*i - 1) = key;
        keys_block(2*i) = ' ';
    end

    % 打印每块结果
    fprintf("Block %d: %s\n", b, join(string(keys_block), ' '));
end
```

运行结果:

```
===== 解码结果 =====
Block 1: 0 6 9 6 6 7 5 3 5 6
Block 2: 4 6 4 6 4 1 5 1 8 0
Block 3: 2 3 3 6 7 3 1 4 1 6
Block 4: 3 6 0 8 3 3 8 1 6 0
Block 5: 4 4 0 0 8 2 6 1 4 6
Block 6: 6 2 5 3 6 8 9 6 3 8
Block 7: 8 4 8 2 1 3 8 1 7 8
Block 8: 5 0 7 3 6 4 3 3 9 9
```

### (3) Goertzel's Algorithm

**Goertzel 算法**是一种高效的频率检测算法，特别适用于识别特定频率成分。

给定复数列  $f[0], f[1], \dots, f[N-1]$

其**离散 Fourier 变换** (Discrete Fourier Transformation, DFT):

$$\tilde{f}_{\text{DFT}}[k] := \sum_{j=0}^{N-1} f[j] \exp(-\frac{i2\pi jk}{N}) \quad (k = 0, 1, \dots, N-1)$$

Goertzel 算法针对某一特定频率索引  $p$  (对应频率  $\frac{p}{N}$  个采样率倍数),

通过以下递推式计算中间序列  $s[k]$ :

$$s[k] = f[k] + 2 \cos(\frac{2\pi p}{N}) s[k-1] - s[k-2]$$

where  $k = 0, 1, \dots, N-1$  and  $s[-2] = s[-1] = 0$

进而可得第  $p$  个 DFT 输出:

$$\tilde{f}_{\text{DFT}}[p] = s[N-1] - \exp(-\frac{i2\pi p}{N}) s[N-2]$$

目标频率分量的幅度为:

$$|\tilde{f}_{\text{DFT}}[p]| = \sqrt{(s[N-2])^2 + (s[N-1])^2 - 2 \cos(\frac{2\pi p}{N}) s[N-2] s[N-1]}$$

根据该幅度的大小，就可判断信号中是否显著含有第  $p$  个频率成分。

若要覆盖整个频谱，则 Goertzel 算法需要对每个  $p = 0, 1, \dots, N-1$  重复上述递推，

复杂度高于快速 Fourier 变换算法。

但若只需计算少量选定的频率分量，它避免了对全谱的冗余计算，在数值上更为高效。

```
% ===== Goertzel 功率计算函数 =====  
function P = goertzel_power(x, fs, target_f)  
    % x      -- 输入信号向量，长度 N (最好是 2 的幂)  
    % fs     -- 采样率  
    % target_f -- 目标频率 (Hz)  
    N = length(x);  
    k = round(target_f * N / fs);          % 最近整数 bin  
    omega = 2*pi*k/N;  
    coeff = 2*cos(omega);  
    s_prev = 0;  
    s_prev2 = 0;  
    for n = 1:N  
        s = x(n) + coeff*s_prev - s_prev2;  
        s_prev2 = s_prev;  
        s_prev = s;  
    end  
    % 最终功率  
    P = s_prev2^2 + s_prev^2 - coeff * s_prev * s_prev2;  
end
```

函数调用:

```
% ===== 解码每个信号块 =====  
% 逐块解码并统一输出
```

```

fprintf("\n===== 解码结果 =====\n");
for b = 1:numBlocks
    segment = signal(block(b,1):block(b,2));
    seg_starts = linspace(1, length(segment), numPerBlock+1);

    % 预分配: 每个按键+空格 共2字符
    keys_block = repmat(' ', 1, numPerBlock*2);

    for i = 1:numPerBlock
        idx1 = floor(seg_starts(i));
        idx2 = floor(seg_starts(i+1)) - 1;
        tone = segment(idx1:idx2);

        % 补零至 2 的幂长
        N = 2^nextpow2(length(tone));
        tone_pad = [tone; zeros(N - length(tone), 1)];

        % 对每个低频计算 Goertzel 功率
        P_low = zeros(size(low_freqs));
        for m = 1:length(low_freqs)
            P_low(m) = goertzel_power(tone_pad, fs, low_freqs(m));
        end
        [~, low_idx_found] = max(P_low);

        % 对每个高频计算 Goertzel 功率
        P_high = zeros(size(high_freqs));
        for m = 1:length(high_freqs)
            P_high(m) = goertzel_power(tone_pad, fs, high_freqs(m));
        end
        [~, high_idx_found] = max(P_high);

        % 找到按键字符并写入预分配数组
        key = dtmf_keys(low_idx_found, high_idx_found);
        keys_block(2*i - 1) = key;
        keys_block(2*i)     = ' ';
    end

    % 打印本块的 10 个按键 (去掉末尾空格)
    fprintf("Block %d: %s\n", b, strtrim(keys_block));
end

```

运行结果:

```

===== 解码结果 =====
Block 1: 0 6 9 6 6 7 5 3 5 6
Block 2: 4 6 4 6 4 1 5 1 8 0
Block 3: 2 3 3 6 7 3 1 4 1 6
Block 4: 3 6 0 8 3 3 8 1 6 0
Block 5: 4 4 0 0 8 2 6 1 4 6
Block 6: 6 2 5 3 6 8 9 6 3 8
Block 7: 8 4 8 2 1 3 8 1 7 8
Block 8: 5 0 7 3 6 4 3 3 9 9

```

## Problem 6 (optional)

---

Show that all eigenvalues of the unitary DFT matrix  $n^{-1/2}F_n$  belong to the set  $\{1, -1, i, -i\}$ .

## Problem 7 (optional)

---

Implement the radix-3 FFT.

A non-recursive implementation is preferred.

Make sure that the complexity of your implementation is  $\Theta(N \log(N))$ .

## Problem 8 (optional)

---

$$V_n = \begin{bmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \dots & \omega^{0 \cdot (n-1)} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \dots & \omega^{1 \cdot (n-1)} \\ \vdots & \vdots & \vdots & \vdots \\ \omega^{(n-1) \cdot 0} & \omega^{(n-1) \cdot 1} & \dots & \omega^{(n-1) \cdot (n-1)} \end{bmatrix}$$

$$\text{where } \omega := \exp\left(-\frac{i2\pi}{n}\right)$$

$$\text{and } (V_n)_{j,k} = \exp\left(-\frac{i2\pi jk}{n}\right) = \omega^{jk}$$

$$V_n^2 = n \begin{bmatrix} 1 & & & 0 \\ & & 0 & 1 \\ & \dots & \dots & \\ 0 & 0 & 1 & \\ 0 & 1 & & \end{bmatrix}$$

6

6. (optional) Show that all eigenvalues of the unitary DFT matrix  $n^{-1/2}F_n$  belong to the set  $\{1, -1, i, -i\}$ .

解: 由第2题可知:  $(n^{-1/2}F_n) \cdot (n^{-1/2}F_n) = J_n$

$$\text{其中 } J_n = \begin{bmatrix} 1 & & & 0 \\ & \ddots & & \\ 0 & & 1 & \\ & & & 0 \end{bmatrix}$$

设  $\lambda$  是  $n^{-1/2}F_n$  的特征值,  $x$  是对应特征向量:

(为方便, 下面的下标编号是从 0 开始, 到  $n-1$  为止)

$$n^{-1/2}F_n x = \lambda x$$

$$\Rightarrow \frac{1}{n} F_n \cdot F_n x = \lambda^2 x$$

$$\Rightarrow J_n \cdot x = \lambda^2 x$$

$$\text{记 } x = (x_0, x_1, \dots, x_{n-1})^T$$

$$\text{若 } x_0 \neq 0, \text{ 则 } x_0 = \lambda^2 x_0 \Rightarrow \lambda \in \{1, -1\}$$

若  $x_0 = 0$ , 由于  $x$  为非零向量, 必有某个分量

$x_k \neq 0$  ( $k > 0$ ), 那么:

$$\begin{cases} x_{n-k} = \lambda^2 x_k & (1) \\ x_k = \lambda^2 x_{n-k} & (2) \end{cases}$$

$$(1) \text{ 代入 } (2), \text{ 得 } x_k = \lambda^2 \cdot \lambda^2 \cdot x_k$$

$$\Rightarrow x_k = \lambda^4 x_k$$

$$\text{由于 } x_k \neq 0 \Rightarrow \lambda \notin \{1, -1, i, -i\}$$

综上所述,  $\lambda \in i, -1, 1, -i$   $\square$

### Problem 9 (optional)

离散卷积定理:

1. Derive the circular convolution theorem based on the following convention of DFT:

$$\hat{u}_k = \sum_{j=0}^{n-1} \exp\left(-\frac{2jk\pi i}{n}\right) u_j, \quad (u \in \mathbb{C}^n).$$

解: 设周期为  $n$ , 则  $u_{k+rn} = u_k, \quad r = \pm 1, \pm 2, \dots$

$$v_{k+rn} = v_k, \quad r = \pm 1, \pm 2, \dots$$

$$\exp\left(\frac{-2(j+rn)k\pi i}{n}\right) = \exp\left(\frac{-2jk\pi i}{n}\right), \quad r = \pm 1, \pm 2, \dots$$

$$(u * v)_j = \sum_{p+q=j} u_p \cdot v_q$$

$$= \sum_{p=0}^{n-1} u_p \cdot v_{j-p} \quad \left( \begin{array}{l} \text{"u" 表示 Fourier 变换} \\ \text{"v" 表示 逆 Fourier 变换} \end{array} \right)$$

$$\widehat{(u * v)}_k = \sum_{j=0}^{n-1} \exp\left(\frac{-2jk\pi i}{n}\right) \cdot (u * v)_j$$

$$= \sum_{j=0}^{n-1} \exp\left(\frac{-2jk\pi i}{n}\right) \cdot \sum_{p=0}^{n-1} u_p \cdot v_{j-p}$$

$$= \sum_{p=0}^{n-1} \cdot \sum_{j=0}^{n-1} \exp\left(\frac{-2(j-p)k\pi i}{n}\right) \cdot v_{j-p} \cdot \exp\left(\frac{-2pk\pi i}{n}\right) \cdot u_p$$

$$\text{注意到: } \sum_{j=0}^{n-1} \exp\left(\frac{-2(j-p)k\pi i}{n}\right) \cdot v_{j-p}$$

$$= \left( \sum_{j=0}^{p-1} + \sum_{j=p}^{n-1} \right) \exp\left(\frac{-2(j-p)k\pi i}{n}\right) \cdot v_{j-p}$$

$$= \left( \sum_{j=0}^{p-1+n} + \sum_{j=p}^{n-1} \right) \exp\left(\frac{-2(j-p)k\pi i}{n}\right) \cdot v_{j-p}$$

$$= \sum_{s=0}^{n-1} \cdot \exp\left(\frac{-2sk\pi i}{n}\right) \cdot V_s$$

R):

$$(\hat{U} * \hat{V})_k = \sum_{p=0}^{n-1} \cdot \sum_{j=0}^{n-1} \cdot \exp\left(\frac{-2(j-p)k\pi i}{n}\right) \cdot V_{j-p} \cdot \exp\left(\frac{-2pk\pi i}{n}\right) \cdot U_p$$

$$= \sum_{p=0}^{n-1} \cdot \sum_{s=0}^{n-1} \cdot \exp\left(\frac{-2sk\pi i}{n}\right) \cdot V_s \cdot \exp\left(\frac{-2pk\pi i}{n}\right) U_p$$

$$= \sum_{s=0}^{n-1} \cdot \exp\left(\frac{-2sk\pi i}{n}\right) \cdot V_s \cdot \sum_{p=0}^{n-1} \cdot \exp\left(\frac{-2pk\pi i}{n}\right) \cdot U_p$$

$$= \hat{V}_k \cdot \hat{U}_k$$

反过来: 若  $\hat{U}$  与  $\hat{V}$  做卷积:

$$(\hat{U} * \hat{V})_k$$

$$= \sum_{p+q=k} \hat{U}_p \cdot \hat{V}_q$$

$$= \sum_{p=0}^{n-1} \hat{U}_p \cdot \hat{V}_{k-p}$$

$$= \sum_{p=0}^{n-1} \cdot \left[ \sum_{j=0}^{n-1} \exp\left(\frac{-2jp\pi i}{n}\right) \cdot U_j \right] \cdot \left[ \sum_{s=0}^{n-1} \exp\left(\frac{-2s(k-p)\pi i}{n}\right) V_s \right]$$



$$= \sum_{j=0}^{n-1} \sum_{s=0}^{n-1} \sum_{p=0}^{n-1} U_j \cdot V_s \cdot \exp\left(\frac{-2sk\pi i}{n}\right) \cdot \exp\left(\frac{-2(j-s)p\pi i}{n}\right)$$

$$= \sum_{j=0}^{n-1} \sum_{s=0}^{n-1} U_j \cdot V_s \cdot \exp\left(\frac{-2sk\pi i}{n}\right) \cdot \sum_{p=0}^{n-1} \exp\left(\frac{-2(j-s)p\pi i}{n}\right)$$

$$= \sum_{j=0}^{n-1} U_j \cdot V_j \cdot \exp\left(\frac{-2jk\pi i}{n}\right) \cdot n$$

$$= n \cdot (\hat{u} \cdot \hat{v})$$

也即：

$$\frac{(\hat{u} * \hat{v})}{n} = u \cdot v$$

□



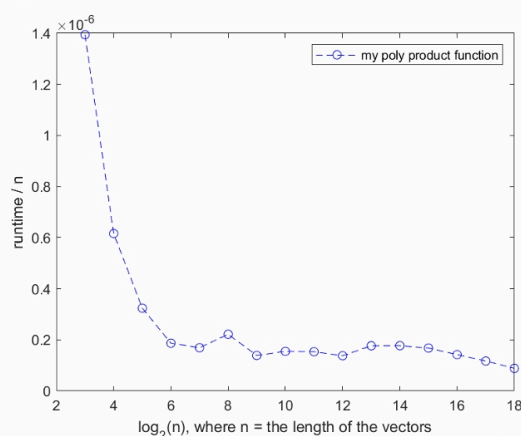
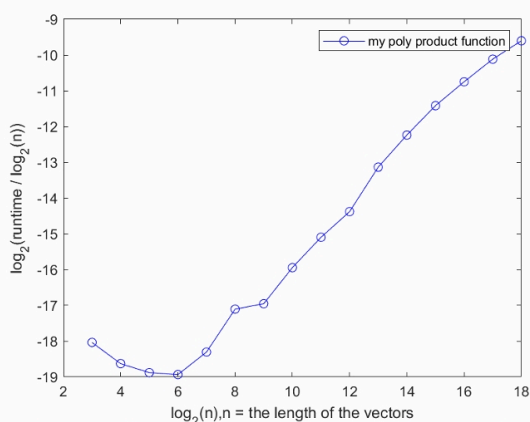
**Problem 10 (optional)**

---

2

2. Write a program to compute the product of two complex polynomials using fast convolution algorithms. You can make use of the MATLAB/Octave function `conv` to check the correctness of your implementation. Make a plot to demonstrate that the complexity of your implementation is  $\Theta(n \log n)$ . What is the complexity of `conv`?

我写的复多项式乘法函数:



左图相当于对  $\log(t/\log n) \sim \log n$  作图

大致是线性的, 斜率大致为1, 说明复杂度至少是  $\Theta(n)$

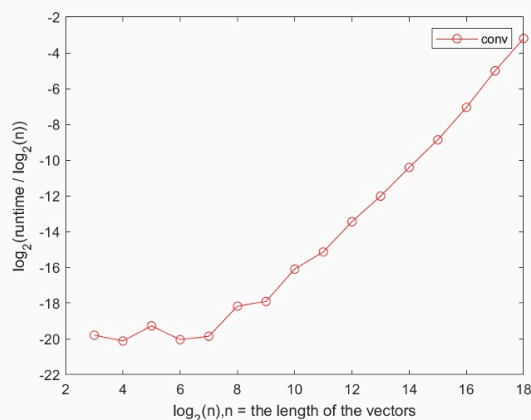
右图相当于对  $t/n \sim \log n$  作图 预期会出现一条

斜率大于0的直线, 但结果与预期不符, 不知什么原因?

Q: 我舍友做出来也是!

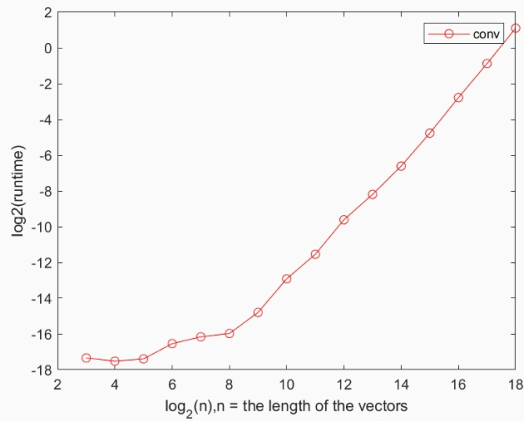
难道比  $\Theta(n \log n)$  还快?

Matlab 的内置 `conv`:



注意到对  $\log(t/\log n) \sim \log n$  作图 斜率大致为2  
说明复杂度高于  $\Theta(n)$

再作  $\log(t) \sim \log(n)$  如下:



斜率仍大致为 2, 故说明 conv 的复杂度在  $\Theta(n^2)$

世那 Matlab 的 conv 应该是直接一项一项地暴力展开的 口

## Problem 11 (optional)

(一维 Poisson 问题系数矩阵的特征值和特征向量)

4. Compute all eigenvalues and eigenvectors of the following  $n \times n$  matrix:

4

$$T_n = \begin{bmatrix} 2 & -1 & & \\ -1 & 2 & & \\ & & \ddots & \\ & & & -1 & 2 & -1 \\ & & & -1 & 2 \end{bmatrix}$$

解: 三对角矩阵的行列式为

$$\det \begin{bmatrix} a & b & & \\ & c & & \\ & & \ddots & \\ & & & c & a \end{bmatrix}_{n \times n} = \begin{cases} \frac{\alpha^{n+1} - \beta^{n+1}}{\alpha - \beta} & \text{if } a^2 \neq 4bc \\ (n+1) \cdot \left(\frac{a}{2}\right)^n & \text{if } a^2 = 4bc \end{cases}$$

其中  $\alpha, \beta$  分别是

$$x^2 - ax + bc = 0 \text{ 的两根}$$

对于  $T_n$ , 只需要求  $T_n - \lambda I$  的特征值与特征向量即可

$$T_n - \lambda I = \begin{bmatrix} 2-\lambda & -1 & & \\ -1 & 2-\lambda & & \\ & & \ddots & \\ & & & -1 & 2-\lambda \end{bmatrix}$$

那么: 对  $|\lambda I - T_n - \lambda I|$ :

$$x^2 - \lambda x + 1 = 0$$

两根分别为:

$$\alpha = \frac{\lambda + \sqrt{\lambda^2 - 4}}{2} \quad \beta = \frac{\lambda - \sqrt{\lambda^2 - 4}}{2}$$

若  $\alpha = \beta$ , 则  $\lambda = 2$  or  $\lambda = -2$  时, 显然  $|\lambda I - T_n - \lambda I| \neq 0$

也即  $\lambda = 2$  or  $\lambda = -2$  不是  $T_n - \lambda I$  特征值

$$\text{若 } \alpha \neq \beta, \text{ 则 } |\lambda I - T_n - \lambda I| = \frac{\alpha^{n+1} - \beta^{n+1}}{\alpha - \beta}$$

$$\Rightarrow \alpha^{n+1} - \beta^{n+1} = 0$$

$$\text{又有 } \alpha \cdot \beta = 1$$

$$\Rightarrow (\alpha \cdot \beta)^{n+1} = \alpha^{2n+2} = 1$$

$$\Rightarrow \alpha = e^{\frac{2\pi i k}{2n+2}} = e^{\frac{k\pi i}{n+1}}, \quad k = 0, \pm 1, \dots, \pm n$$

$$\text{又注意到 } \alpha = \bar{\beta} \text{ 并且 } \alpha \neq \beta$$

$$\text{则 } \lambda = \alpha + \beta = 2 \cos \frac{k\pi}{n+1}, \quad k = 1, 2, \dots, n$$

$$\Rightarrow \underline{T_n \text{ 的特征值为 } \lambda + 2}$$

$$\text{也即 } \underline{2 \cos \frac{k\pi}{n+1} + 2}, \quad k = 1, 2, \dots, n$$

再看特征向量的情况:  $T_n - 2I$  与  $T_n$  的特征向量是相同的

$$\text{若 } \lambda = 2 \cos \frac{k\pi}{n+1}$$

$$\text{取 } x = \left( \sin \frac{k\pi}{n+1}, \sin \frac{2k\pi}{n+1}, \dots, \sin \frac{n k \pi}{n+1} \right)^T$$

注意到

$$\begin{cases} 2 \cos \frac{k\pi}{n+1} \cdot \sin \frac{k\pi}{n+1} - \sin \frac{2k\pi}{n+1} = 0 \\ 2 \cos \frac{k\pi}{n+1} \cdot \sin \frac{j k \pi}{n+1} - \sin \frac{(j-1) k \pi}{n+1} - \sin \frac{(j+1) k \pi}{n+1} = 0 \\ - \sin \frac{(j-1) k \pi}{n+1} + 2 \cos \frac{k\pi}{n+1} \cdot \sin \frac{j k \pi}{n+1} - \sin \frac{(j+1) k \pi}{n+1} = 0 \end{cases}$$

(通过和差化积公式验证)

则:

$$\begin{bmatrix} \lambda - 1 & & & \\ & \ddots & & \\ & & \lambda & \\ & & & \ddots \\ & & & & \lambda - 1 \end{bmatrix} \cdot x = 0$$

$\Rightarrow$   $\lambda$  是  $-\lambda$ , 也即  $\lambda = 2 \cos \frac{(n+1-k)\pi}{n+1}$  的特征向量

$\therefore T_n$  的特征向量为:

$$x = \left( \sin \frac{k\pi}{n+1}, \sin \frac{2k\pi}{n+1}, \dots, \sin \frac{nk\pi}{n+1} \right)^T$$

其中  $k = 1, 2, \dots, n$

Q: 这题做得要吐血! 有无简单妙法?

The End